

Netty解决粘包半包问题（下）

北京理工大学计算机学院
金旭亮



概述



前一讲我们介绍了TCP开发中必然会遇到的粘包与半包问题，并且介绍了Netty内置了若干个Handler，采用相应的策略来解决这个问题。



本讲我们通过实例，介绍如何使用Handler，解决粘包与半包问题。

固定数据传输的大小

一种最简单的方案，就是事先约定，在客户端与服务端之间，只传送固定大小的数据。

```
public class MyClientSendHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        //这个ByteBuf在多次发送数据时，被重用
        ByteBuf buf = ctx.alloc().buffer();
        //分100次发送一个整数到服务端
        for (int i = 0; i < 100; i++) {
            //总是从第一个字节开始写
            buf.clear();
            buf.writeInt(i);
            //通知Netty，这个ByteBuf在数据发送期间需要保留
            buf.retain();
            System.out.println("发送: " + i);
            ctx.writeAndFlush(buf);
        }
        //释放ByteBuf
        buf.release();
    }
}
```

在这个客户端Handler中，分多次将一个整数发给服务端。

服务端Handler

服务端每次从收到的ByteBuf中读取4个字节，装配为一个int：

```
public class MyServerHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("有客户端连接: "+ctx.channel().remoteAddress());
    }
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {
        ByteBuf buf = (ByteBuf) msg;
        //一个int类型占4个字节
        System.out.println("收到数字: "+buf.readInt());
    }
}
```

设定服务端管线

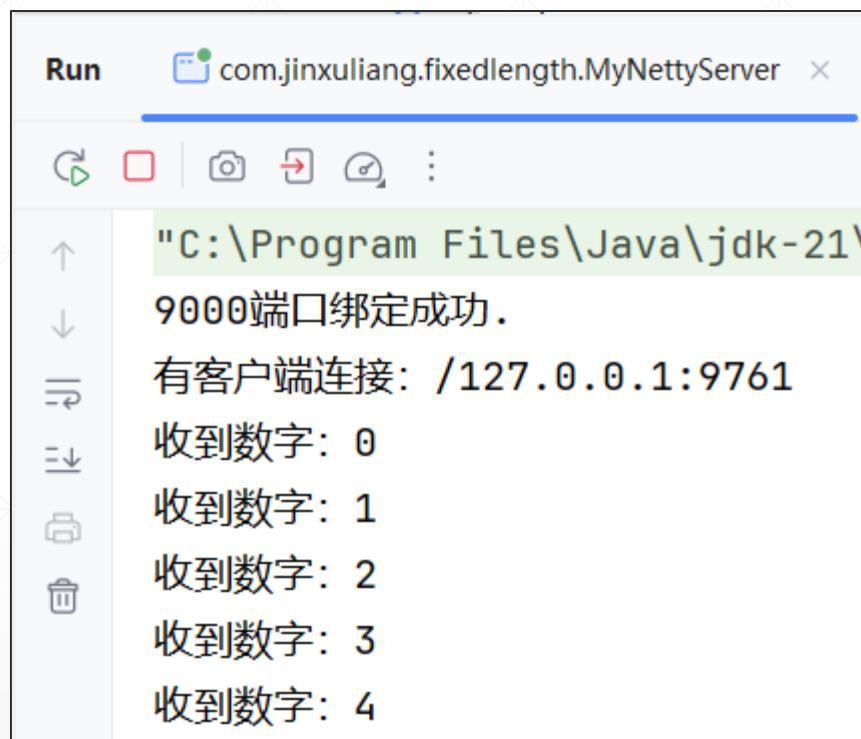
Netty提供了一个FixedLengthFrameDecoder，用于指定将多少个字节的数据合并到一个ByteBuf中，传给后面的Handler。

```
ServerBootstrap boot = new ServerBootstrap();
boot.group(boosGroup,workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        @Override
        protected void initChannel(SocketChannel socketChannel) throws Exception {
            //每次读取4个字节，因为一个int数据，占据4个字节，如果改为5，将会解码错误。
            socketChannel.pipeline().addLast(new FixedLengthFrameDecoder(4));
            socketChannel.pipeline().addLast(new MyServerHandler());
        }
    });
```

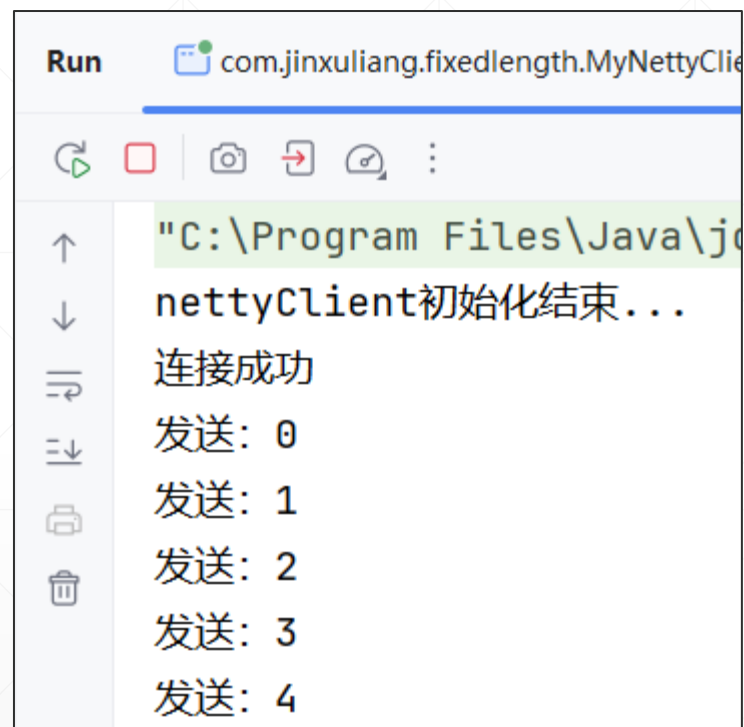


上述代码中，设定FixedLengthFrameDecoder按4个字节一组写入ByteBuf，一会儿你可以测试一下，改为5个字节，或者干脆注释掉这一句，看看结果怎么样.....

正常状况



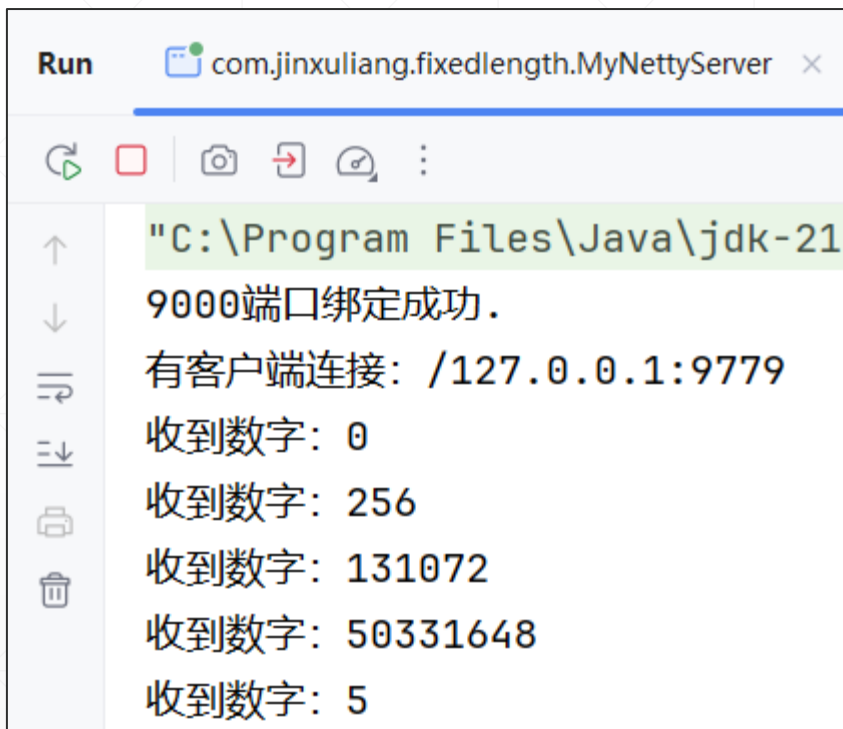
```
Run com.jinxuliang.fixedlength.MyNettyServer x
"C:\Program Files\Java\jdk-21\
9000端口绑定成功.
有客户端连接: /127.0.0.1:9761
收到数字: 0
收到数字: 1
收到数字: 2
收到数字: 3
收到数字: 4
```



```
Run com.jinxuliang.fixedlength.MyNettyClient
"C:\Program Files\Java\jdk-21\
nettyClient初始化结束...
连接成功
发送: 0
发送: 1
发送: 2
发送: 3
发送: 4
```

出错情况

`new FixedLengthFrameDecoder(5)`



The screenshot shows an IDE console window titled "Run" for the class `com.jinxuliang.fixedlength.MyNettyServer`. The console output is as follows:

```
"C:\Program Files\Java\jdk-21\
9000端口绑定成功.
有客户端连接: /127.0.0.1:9779
收到数字: 0
收到数字: 256
收到数字: 131072
收到数字: 50331648
收到数字: 5
```

`FixedLengthFrameDecoder`那句代码被注释掉.....



The screenshot shows an IDE console window titled "Run" for the class `com.jinxuliang.fixedlength.MyNettyServer`. The console output is as follows:

```
"C:\Program Files\Java\jdk-21\
9000端口绑定成功.
有客户端连接: /127.0.0.1:9797
收到数字: 0
```

客户端按4字节发送数据，服务端按5字节接收，结果，收到的数据是错的。

客户端按4字节发送数据，服务端没有使用 `FixedLengthFrameDecoder`，结果只收到了第1个数字，后面的全没读出来！



强制约定客户端与服务端传送数据的字节数，虽然实现起来简单，但很明显过于僵化，无法适应需要传送变长信息的场景。

如果需要传送变长信息，又该怎么办呢？

LineBasedFrameDecoder



最简单的一种方法，就是要求客户端发来的所有信息，都以“行”为基本单位。



区分行的基本方法，就是在收到的字符串中通过查找“\n”或“\n”来识别消息的完整性。



这一解码器，默认使用UTF-8解码收到的数据。



使用这个解码器，只需要在服务端将其加入到pipeline中即可。



当然，客户端在发送消息时，必须在末尾加入“\n”。

客户端发送数据

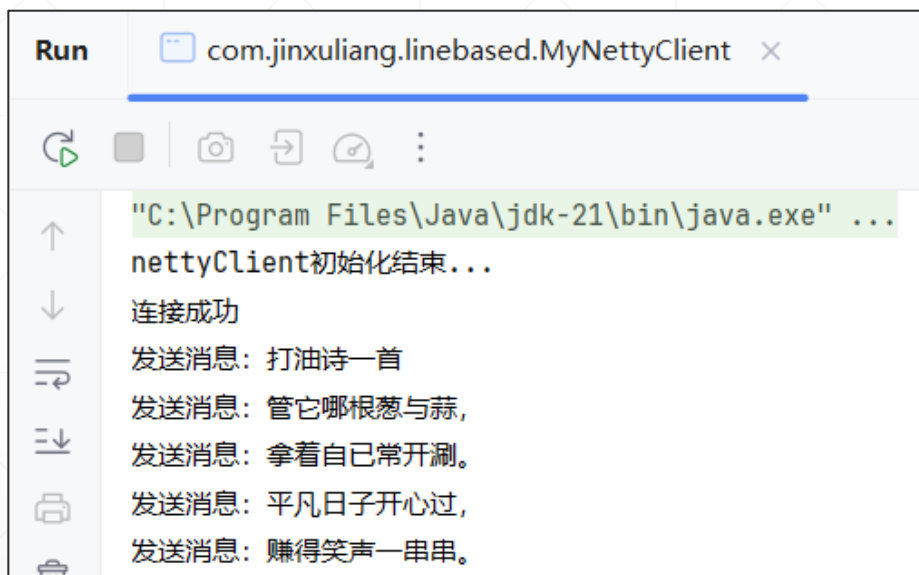
```
private void sendMessage(String msg) {  
    System.out.println("发送消息: " + msg);  
    //用“\n”作为消息分隔符  
    msg = msg + "\n";  
    //将一个字符串写入到ByteBuf中  
    ByteBuf buf = ctx.alloc().buffer();  
    var messageData = msg.getBytes(StandardCharsets.UTF_8);  
    buf.writeBytes(messageData);  
    //将数据发送到服务端  
    ctx.writeAndFlush(buf);  
}
```

服务端引入LineBasedFrameDecoder处理器:

```
ServerBootstrap boot = new ServerBootstrap();  
boot.group(boosGroup,workerGroup)  
    .channel(NioServerSocketChannel.class)  
    .childHandler(new ChannelInitializer<SocketChannel>() {  
        @Override  
        protected void initChannel(SocketChannel socketChannel) throws Exception {  
            socketChannel.pipeline().addLast(new LineBasedFrameDecoder(1024));  
            socketChannel.pipeline().addLast(new MyServerHandler());  
        }  
    });
```

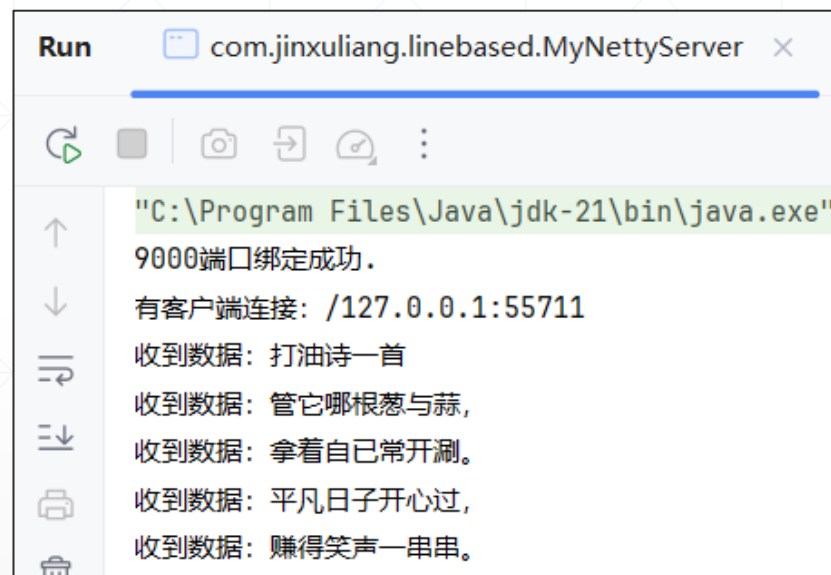
运行截图

客户端



```
Run com.jinxuliang.linebased.MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
nettyClient初始化结束...
连接成功
发送消息: 打油诗一首
发送消息: 管它哪根葱与蒜,
发送消息: 拿着自己常开涮。
发送消息: 平凡日子开心过,
发送消息: 赚得笑声一串串。
```

服务端



```
Run com.jinxuliang.linebased.MyNettyServer x
"C:\Program Files\Java\jdk-21\bin\java.exe"
9000端口绑定成功.
有客户端连接: /127.0.0.1:55711
收到数据: 打油诗一首
收到数据: 管它哪根葱与蒜,
收到数据: 拿着自己常开涮。
收到数据: 平凡日子开心过,
收到数据: 赚得笑声一串串。
```



从程序运行结果来看，加入了LineBasedFrameDecoder之后，服务端的“粘包”和“半包”现象，再未出现.....

自定义消息分割符

如果不想使用“\n”或“\r\n”作为消息分割符，服务端可以使用DelimiterBasedFrameDecoder这个处理器，指定特定的字符作为分割符。

```
public static final String SEPARATOR="||";
```

这时，客户端发送消息时，必须使用此分割符分割消息：

```
private void sendMessage(String msg) {  
    System.out.println("发送消息: " + msg);  
    //用“||”作为消息分隔符  
    msg = msg + MyNettyClient.SEPARATOR;  
    //将一个字符串写入到ByteBuf中  
    ByteBuf buf = ctx.alloc().buffer();  
    var messageData = msg.getBytes(StandardCharsets.UTF_8);  
    buf.writeBytes(messageData);  
    //将数据发送到服务端  
    ctx.writeAndFlush(buf);  
}
```

服务端加入DelimiterBasedFrameDecoder

```
ServerBootstrap boot = new ServerBootstrap();
boot.group(boosGroup,workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        @Override
        protected void initChannel(SocketChannel socketChannel) throws Exception {
            socketChannel.pipeline().addLast(new DelimiterBasedFrameDecoder(1024,
                Unpooled.copiedBuffer(MyNettyClient.SEPARATOR.getBytes(StandardCharsets.UTF_8))));
            socketChannel.pipeline().addLast(new MyServerHandler());
        }
    });
```

完成这个工作之后，客户端发来的消息，就可以被服务端成功解析了。

基于长度的消息分割

如果客户端在发送一条消息时，给消息附加上一条额外信息，指明“本消息的长度”，那么服务端依据这个信息，就能成功地得到一条完整的消息：

消息长度（4个字节）	消息本体
------------	------

下面我们依据这个想法，编写一个示例，实现客户端与服务端之间的“变长消息交换”。

客户端消息发送-1

```
public class MyClientSendPrefixMessageHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println(new Date() + ":客户端发送数据");
        // 将一个字符串写入到ByteBuf中
        ByteBuf buf = ctx.alloc().buffer();
        writeMessage(buf, "白日依山尽");
        writeMessage(buf, "黄河入海流");
        writeMessage(buf, "欲穷千里目");
        writeMessage(buf, "更上一层楼");
        // 将数据发送到服务端
        ctx.writeAndFlush(buf);
    }

    private static void writeMessage(ByteBuf buf, String message) {
        var messageData = message.getBytes(StandardCharsets.UTF_8);
        System.out.println("长度" + messageData.length);
        buf.writeInt(messageData.length);
        buf.writeBytes(messageData);
    }
}
```

客户端消息发送-2

```
NioEventLoopGroup workerGroup = new NioEventLoopGroup();
Bootstrap bootstrap = new Bootstrap();
bootstrap.group(workerGroup)
    .channel(NioSocketChannel.class)
    .handler(new ChannelInitializer<SocketChannel>() {
        @Override
        protected void initChannel(SocketChannel ch) throws Exception {
            //添加handler将数据发送到Server端
            ch.pipeline().addLast(new MyClientSendPrefixMessageHandler());
        }
    });
```


服务端代码

服务端解码器

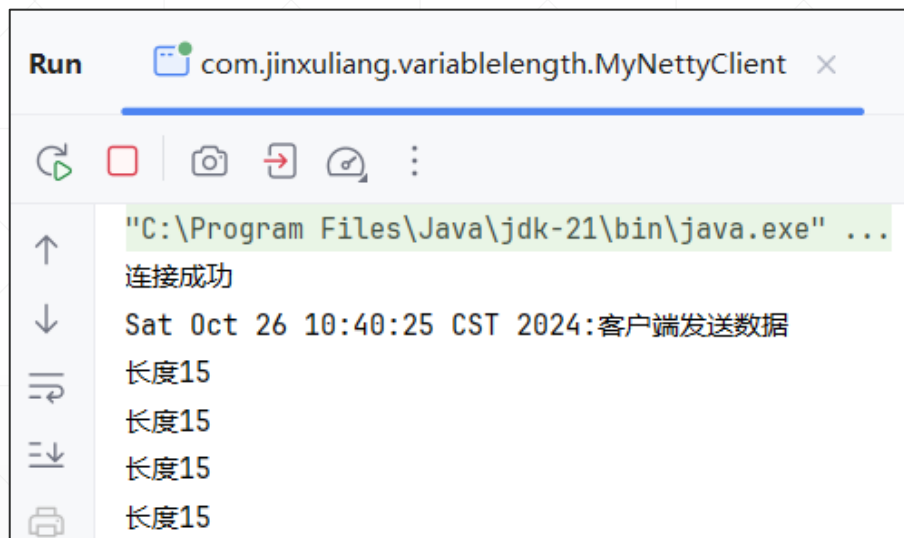
```
//期望消息格式: 4字节消息长度|真实消息数据
public class PrefixLengthDecoder extends ByteToMessageDecoder {
    @Override
    protected void decode(ChannelHandlerContext channelHandlerContext,
                          ByteBuf byteBuf, List<Object> list) throws Exception {
        if(byteBuf.readableBytes()<4){
            return;
        }
        int length = byteBuf.readInt();
        System.out.println("length:"+length);
        //只有等到收到了完整的数据,才开始运行
        if(byteBuf.readableBytes()<length){
            byteBuf.resetReaderIndex();
            return;
        }
        byte[] data=new byte[length];
        byteBuf.readBytes(data);
        System.out.println(new String(data));
        //记下当前位置,以便从这里继续读
        //这时假设Client会发送多条数据。
        byteBuf.markReaderIndex();
    }
}
```

服务端管线

```
ServerBootstrap boot = new ServerBootstrap();
boot.group(boosGroup,workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        @Override
        protected void initChannel(SocketChannel socketChannel) throws Exception {
            socketChannel.pipeline().addLast(new PrefixLengthDecoder());
        }
    });
```

运行截图

客户端



The screenshot shows the console output of the client application. The title bar indicates the class is `com.jinxuliang.variablelength.MyNettyClient`. The output shows a successful connection, followed by a timestamped log entry indicating data was sent from the client. Subsequent lines show the received data length as 15 bytes four times.

```
Run com.jinxuliang.variablelength.MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功
Sat Oct 26 10:40:25 CST 2024:客户端发送数据
长度15
长度15
长度15
长度15
```

服务端



The screenshot shows the console output of the server application. The title bar indicates the class is `com.jinxuliang.variablelength.MyNettyServer`. The output shows the server successfully binding to port 9000. It then receives data from the client, which is decoded as the poem '白日依山尽，黄河入海流。欲穷千里目，更上一层楼'.

```
Run com.jinxuliang.variablelength.MyNettyServer x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
length:15
白日依山尽
length:15
黄河入海流
length:15
欲穷千里目
length:15
更上一层楼
```

LengthFieldBasedFrameDecoder

基于长度切分消息，在实际开发中有广泛应用，固然我们可以像前面那样，手工实现，但Netty提供了一个功能更为完备的解码器——LengthFieldBasedFrameDecoder，推荐在实际开发中使用它。

下面我们通过代码介绍它的基本用法。

客户端发送消息-1

客户端将重复100次发送一首打油诗给服务端，与前面的示例相比，这块代码没有多少变化，唯一需要注意的，就是现在只需发送“纯”信息，无需加入额外的“分割符”。

```
public class MyClientSendHandler extends SimpleChannelInboundHandler<String> {
    private ChannelHandlerContext ctx = null;

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        this.ctx = ctx;
    }

    private void sendMessage(String msg) {
        System.out.println("发送消息: " + msg);
        //将数据发送到服务端
        ctx.writeAndFlush(msg);
    }

    @Override
    protected void channelRead0(ChannelHandlerContext ctx, String msg) {
        System.out.println("收到消息: " + msg);
        for (int i = 1; i <= 100; i++) {
            sendMessage("----" + i + "----");
            sendMessage("打油诗一首");
            sendMessage("管它哪根葱与蒜，");
            sendMessage("拿着自己常开涮。");
            sendMessage("平凡日子开心过，");
            sendMessage("赚得笑声一串串。");
        }
    }
}
```

客户端发送消息-2

从代码中可以看到，管线中加入了LengthFieldBasedFrameDecoder用于解码（按长度切分收到的消息），在管线开头又加入了LengthFieldPrepender用于编码（自动给发出的消息加上长度信息。

另外，管线中还加入了Netty内置的字符串编解码器。

```
NioEventLoopGroup workerGroup = new NioEventLoopGroup();
Bootstrap bootstrap = new Bootstrap();
bootstrap.group(workerGroup)
    .channel(NioSocketChannel.class)
    .handler(new ChannelInitializer<SocketChannel>() {
        @Override no usages
        protected void initChannel(SocketChannel ch) throws Exception {
            //添加handler将数据发送到Server端
            ch.pipeline().addLast(new LengthFieldBasedFrameDecoder(
                maxFrameLength: 102400,
                lengthFieldOffset: 0,
                lengthFieldLength: 2,
                lengthAdjustment: 0,
                initialBytesToStrip: 2));
            ch.pipeline().addLast(new StringDecoder(StandardCharsets.UTF_8));
            ch.pipeline().addLast(new MyClientSendHandler());
            //注意，以下两个Handler是放到开头的
            ch.pipeline().addFirst(new StringEncoder(StandardCharsets.UTF_8));
            ch.pipeline().addFirst(new LengthFieldPrepender( lengthFieldLength: 2));
        }
    });
```

服务端接收消息-1

服务端接收消息的Handler极为简单，如下所示：

```
public class MyServerHandler extends SimpleChannelInboundHandler<String> {  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        System.out.println("有客户端连接: "+ctx.channel().remoteAddress());  
        ctx.writeAndFlush("\n欢迎连接本服务器!\n");  
    }  
  
    @Override  
    protected void channelRead0(ChannelHandlerContext ctx, String msg) throws Exception {  
        System.out.println("收到数据: "+msg);  
    }  
}
```

服务端接收消息-2

服务端管线与客户端管线，几乎是一样的：

```
ServerBootstrap boot = new ServerBootstrap();
boot.group(boosGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        @Override no usages
        protected void initChannel(SocketChannel socketChannel) throws Exception {
            socketChannel.pipeline().addLast(new LengthFieldBasedFrameDecoder(
                maxFrameLength: 102400,
                lengthFieldOffset: 0,
                lengthFieldLength: 2,
                lengthAdjustment: 0,
                initialBytesToStrip: 2));

            socketChannel.pipeline().addLast(new StringDecoder());
            socketChannel.pipeline().addLast(new MyServerHandler());
            //注意，以下两个Handler是放到开头的
            socketChannel.pipeline().addFirst(new StringEncoder());
            socketChannel.pipeline().addFirst(new LengthFieldPrepender( lengthFieldLength: 2));
        }
    });
```

运行截图

